
Week 5: Text Generation using RNN, LSTM and GRU

This notebook trains RNN, LSTM and GRU on five different corpora.

```
In [ ]: import tensorflow as tf
        from tensorflow.keras.preprocessing.text import Tokenizer
        from tensorflow.keras.preprocessing.sequence import pad_sequences
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense
        import numpy as np
        import matplotlib.pyplot as plt
```

Student Learning Tasks

- Embedding Dimension = 64
- Hidden Units = 128
- Epochs = 200
- Generate 10 words

```

In [ ]: def train_models(corpus):
    tokenizer=Tokenizer()
    tokenizer.fit_on_texts([corpus])
    total_words=len(tokenizer.word_index)+1
    seq=[]
    for line in corpus.split("\n"):
        t=tokenizer.texts_to_sequences([line])[0]
        for i in range(1,len(t)):
            seq.append(t[:i+1])
    max_len=max(len(x) for x in seq)
    seq=pad_sequences(seq,maxlen=max_len,padding='pre')
    X=seq[:, :-1]
    y=seq[:, -1]
    def build(kind):
        m=Sequential([Embedding(total_words,64,input_length=max_len-1),
                       kind(128),
                       Dense(total_words,activation='softmax')])
    m.compile(optimizer='adam',loss='sparse_categorical_crossentropy')
    h=m.fit(X,y,epochs=200,verbose=0)
    return m,h
    rnn,rh=build(SimpleRNN)
    lstm,lh=build(LSTM)
    gru,gh=build(GRU)
    return tokenizer,max_len,rnn,lstm,gru,rh,lh,gh

def generate_text(model,tokenizer,max_len,seed,next_words=10):
    for _ in range(next_words):
        t=tokenizer.texts_to_sequences([seed])[0]
        t=pad_sequences([t],maxlen=max_len-1,padding='pre')
        pred=np.argmax(model.predict(t,verbose=0),axis=-1)[0]
        word=""
        for w,i in tokenizer.word_index.items():
            if i==pred:
                word=w
                break
        seed+=" "+word
    return seed

```

Experiment 1: Shakespeare

```
In [ ]: corpus = '''To be or not to be that is the question.  
All the world's a stage and all the men and women merely players.  
The course of true love never did run smooth.'''  
tokenizer,max_len,rnn,lstm,gru,rh,lh,gh=train_models(corpus)  
  
plt.figure(figsize=(8,4))  
plt.plot(rh.history['loss'],label='RNN')  
plt.plot(lh.history['loss'],label='LSTM')  
plt.plot(gh.history['loss'],label='GRU')  
plt.legend()  
plt.show()  
  
print(generate_text(rnn,tokenizer,max_len,'to be',10))  
print(generate_text(lstm,tokenizer,max_len,'to be',10))  
print(generate_text(gru,tokenizer,max_len,'to be',10))
```

Experiment 2: Song Lyrics

```
In [ ]: corpus = '''The morning sun is shining bright.  
Birds are singing in the light.  
Dreams are flying like a kite.'''  
tokenizer,max_len,rnn,lstm,gru,rh,lh,gh=train_models(corpus)  
  
plt.figure(figsize=(8,4))  
plt.plot(rh.history['loss'],label='RNN')  
plt.plot(lh.history['loss'],label='LSTM')  
plt.plot(gh.history['loss'],label='GRU')  
plt.legend()  
plt.show()  
  
print(generate_text(rnn,tokenizer,max_len,'the morning',10))  
print(generate_text(lstm,tokenizer,max_len,'the morning',10))  
print(generate_text(gru,tokenizer,max_len,'the morning',10))
```

Experiment 3: Chatbot

```
In [ ]: corpus = '''User hello
Bot hello how can I help you today
User tell me about artificial intelligence
Bot artificial intelligence learns from data'''
tokenizer,max_len,rnn,lstm,gru,rh,lh,gh=train_models(corpus)

plt.figure(figsize=(8,4))
plt.plot(rh.history['loss'],label='RNN')
plt.plot(lh.history['loss'],label='LSTM')
plt.plot(gh.history['loss'],label='GRU')
plt.legend()
plt.show()

print(generate_text(rnn,tokenizer,max_len,'user hello',10))
print(generate_text(lstm,tokenizer,max_len,'user hello',10))
print(generate_text(gru,tokenizer,max_len,'user hello',10))
```

Experiment 4: Story

```
In [ ]: corpus = '''A young explorer entered an ancient forest.
He found a hidden cave with glowing stones.
The adventure changed his life forever.'''
tokenizer,max_len,rnn,lstm,gru,rh,lh,gh=train_models(corpus)

plt.figure(figsize=(8,4))
plt.plot(rh.history['loss'],label='RNN')
plt.plot(lh.history['loss'],label='LSTM')
plt.plot(gh.history['loss'],label='GRU')
plt.legend()
plt.show()

print(generate_text(rnn,tokenizer,max_len,'a young',10))
print(generate_text(lstm,tokenizer,max_len,'a young',10))
print(generate_text(gru,tokenizer,max_len,'a young',10))
```

Experiment 5: Custom PDF

```
In [ ]: corpus = '''Paste your extracted PDF text here.
Replace this sample with your own PDF content.'''
tokenizer,max_len,rnn,lstm,gru,rh,lh,gh=train_models(corpus)

plt.figure(figsize=(8,4))
plt.plot(rh.history['loss'],label='RNN')
plt.plot(lh.history['loss'],label='LSTM')
plt.plot(gh.history['loss'],label='GRU')
plt.legend()
plt.show()

print(generate_text(rnn,tokenizer,max_len,'paste your',10))
print(generate_text(lstm,tokenizer,max_len,'paste your',10))
print(generate_text(gru,tokenizer,max_len,'paste your',10))
```

Conclusion

- Vanilla RNN learns short patterns but struggles with memory.
- LSTM captures long-range dependencies better.
- GRU provides similar performance with faster training.
- Multiple corpora demonstrate sequence modeling across different styles.

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).